

# PostgreSQL, e le righe che non cambiano

Un gruppo di processi che cooperano, un file di autenticazione che decide chi può connettersi e da dove, e l'MVCC mostrato direttamente — la stessa riga prima e dopo un UPDATE, con gli id di transazione in chiaro. Da un lab acceso.

---

**Comando** `qlab run postgres-lab`

**VM** `postgres-lab`, 5432 su porta host dinamica

**Credenziali** `labuser` / `labpass`

**Esito** `qlab test postgres-lab` → 6 esercizi, tutti superati

# 1. Un cluster sono più processi

```
psql (PostgreSQL) 14.24 (Ubuntu 14.24-0ubuntu0.22.04.1)
active
Ver Cluster Port Status Owner    Data directory          Log file
14  main     5432 online postgres /var/lib/postgresql/14/main /var/log/postgresql/postgresql-14-main.log
```

```
postgres      4832 /usr/lib/postgresql/14/bin/postgres -D /var/lib/postgresql/14/main -c config_file=/etc/
postgresql/14/main/postgresql.conf
postgres      4834 postgres: 14/main: checkpointer
postgres      4835 postgres: 14/main: background writer
postgres      4836 postgres: 14/main: walwriter
postgres      4837 postgres: 14/main: autovacuum launcher
postgres      4838 postgres: 14/main: stats collector
postgres      4839 postgres: 14/main: logical replication launcher
```

Un supervisore e una serie di specialisti, ciascuno chiamato col proprio mestiere. Vale la pena conoscerli per nome, perché sono quello che si vede in `ps` quando qualcosa non va:

- **checkpointer** — scarica le pagine sporche e segna un punto da cui il database può ripartire.
- **walwriter** — scrive il write-ahead log. Ogni modifica viene registrata lì *prima* di toccare i file dati, ed è ciò che rende possibile il recupero dopo un crash.
- **autovacuum launcher** — recupera le versioni di riga morte di cui parla la sezione 4. Lasciato spento, una tabella movimentata cresce senza limite.
- **background writer, stats collector, logical replication launcher.**

Postgres chiama **cluster** un'istanza in esecuzione: un server, una directory dati, molti database. Qui quella parola non significa più macchine.

```
LISTEN 0.0.0.0:5432
LISTEN [::]:5432
```

```

      List of databases
  Name      | Owner   | Encoding | Collate | Ctype   | Access privileges
-----+-----+-----+-----+-----+-----
postgres   | postgres | UTF8     | C.UTF-8 | C.UTF-8 |
template0  | postgres | UTF8     | C.UTF-8 | C.UTF-8 | =c/postgres      +
           |          |          |          |          | postgres=CTc/postgres
template1  | postgres | UTF8     | C.UTF-8 | C.UTF-8 | =c/postgres      +
           |          |          |          |          | postgres=CTc/postgres
testdb     | labuser  | UTF8     | C.UTF-8 | C.UTF-8 | =Tc/labuser      +
           |          |          |          |          | labuser=CTc/labuser
(4 rows)
```

`template0` e `template1` non sono ingombro: un database nuovo è una copia di un template. Tutto ciò che si installa in `template1` comparirà in ogni database creato dopo; `template0` è quello intatto a cui si può sempre tornare.

## 2. Chi può connettersi, da dove e come

```
local all postgres peer
local all all peer
host all all 127.0.0.1/32 md5
host all all ::1/128 md5
local replication all peer
host replication all 127.0.0.1/32 md5
host replication all ::1/128 md5
host all all 0.0.0.0/0 md5
```

`pg_hba.conf` si legge dall'alto in basso e **vince la prima riga che corrisponde**. Ogni riga è: tipo di connessione, database, utente, indirizzo, metodo.

- `local ... peer` — sul socket unix, PostgreSQL chiede al kernel quale utente di sistema siete e lo fa corrispondere a un ruolo con lo stesso nome. Non esiste alcuna password. È per questo che `sudo -u postgres psql` funziona e `psql -U postgres` da un altro utente di sistema no.
- `host ... 127.0.0.1/32 md5` — via TCP, serve una password.

```
$ sudo -u postgres psql -c "SELECT current_user"    (unix socket, peer auth)
postgres|t

$ psql -h 127.0.0.1 -U labuser testdb    (TCP, password auth)
labuser|127.0.0.1
```

Lo stesso server, due connessioni, due identità — e `inet_server_addr()` restituisce NULL sul socket, che è un modo elegante di sapere quale strada si è presa.

### L'ultima riga di quel file

`host all all 0.0.0.0/0 md5` accetta una connessione con password da qualsiasi indirizzo della Terra, verso qualsiasi database, come qualsiasi ruolo. È lì perché si possa puntare un'interfaccia grafica al laboratorio, ed è esattamente la riga da togliere su qualunque cosa di vero.

```

                                List of roles
Role name |                               Attributes                               | Member of
-----+-----+-----+-----+-----+-----+-----+-----+-----+-----+-----+
labuser   | Create DB                               | {}
postgres | Superuser, Create role, Create DB, Replication, Bypass RLS | {}
```

Postgres non distingue fra «utente» e «gruppo»: sono entrambi **ruoli**, e un ruolo che può fare login è ciò che altri sistemi chiamano utente. `postgres` è il superutente; `labuser` può solo creare database.

### 3. Una join, e cosa ne ha fatto il planner

```
$ SELECT u.name, COUNT(o.id) AS orders, SUM(o.amount) AS spent ...
   name  | orders | spent
-----+-----+-----
 Alice   |      2 | 1079.98
 Charlie |      1 | 349.99
 Bob     |      1 | 29.99
(3 rows)
```

```
$ EXPLAIN ANALYZE the same join
                                QUERY PLAN
-----
HashAggregate (cost=29.85..32.47 rows=210 width=154) (actual time=0.030..0.031 rows=3 loops=1)
  Group Key: u.id
  Batches: 1  Memory Usage: 40kB
  -> Hash Right Join (cost=14.72..28.40 rows=290 width=138) (actual time=0.020..0.023 rows=4 loops=1)
    Hash Cond: (o.user_id = u.id)
    -> Seq Scan on orders o (cost=0.00..12.90 rows=290 width=20) (actual time=0.001..0.002 rows=4 loops=1)
    -> Hash (cost=12.10..12.10 rows=210 width=122) (actual time=0.009..0.009 rows=3 loops=1)
      Buckets: 1024  Batches: 1  Memory Usage: 9kB
```

`EXPLAIN ANALYZE` non stima: esegue la query e riferisce cosa è successo davvero, tempi e conteggi reali compresi. Lo scarto fra le righe stimate ( `rows=` ) e quelle effettive ( `actual rows=` ) è la prima cosa da guardare quando una query è lenta: se la stima del planner è molto sbagliata, ha scelto la strategia su informazioni sbagliate, e il rimedio di solito sono le statistiche, non una query diversa.

### 4. L'MVCC, in chiaro

```
Every row carries the transaction that created it and the one that deleted it.
 xmin | xmax | id | name
-----+-----+-----
  737 |   739 |  1 | Alice
  737 |   739 |  2 | Bob
  737 |   739 |  3 | Charlie
(3 rows)
```

```
$ after an UPDATE, xmin changes: the old row version is still there until VACUUM
 xmin | xmax | id | name
-----+-----+-----
  761 |     0 |  1 | Alice
  737 |   739 |  2 | Bob
  737 |   739 |  3 | Charlie
(3 rows)
```

È la decisione di progetto centrale di PostgreSQL, ed eccola allo scoperto.

Ogni riga porta due colonne nascoste. `xmin` è la transazione che ha creato questa versione della riga; `xmax` è quella che l'ha cancellata o bloccata, oppure `0` se nessuna l'ha fatto.

Si guardi la riga 1 prima e dopo l' `UPDATE` . Il suo `xmin` è cambiato. Postgres non ha modificato la riga sul posto: ne ha scritto una **nuova versione**, e la vecchia è ancora su disco, ora invisibile alle nuove transazioni. È per questo che un `UPDATE` può costare quanto un `INSERT` , e per cui i

lettori non bloccano mai gli scrittori: un lettore continua semplicemente a vedere la versione che era corrente quando è partito.

Il prezzo è che le versioni morte si accumulano. Recuperarle è esattamente ciò per cui esistono `VACUUM` e l'autovacuum launcher visto sopra.

## 5. Verifica

---

```
Exercise 1: All 6 checks passed
Exercise 2: All 6 checks passed
Exercise 3: All 5 checks passed
Exercise 4: All 2 checks passed
Exercise 5: All 6 checks passed
Exercise 6: All 7 checks passed
Exercises run:      6
Exercises passed:   6
Exercises failed:   0
Exercises skipped:  0
All exercises passed!
```

## 6. Cosa portarsi via

---

- Un cluster è un server con molti database, non molte macchine.
- `pg_hba.conf` : vince la prima corrispondenza. L'ordine conta più del contenuto.
- `peer` autentica tramite l'identità di sistema sul socket; `md5` vuole una password via TCP. Quasi tutta la confusione del tipo «con sudo funziona, altrimenti no» è questa.
- Utenti e gruppi sono entrambi ruoli.
- `UPDATE` scrive una nuova versione di riga. I lettori non bloccano gli scrittori, e il conto lo paga `VACUUM`.
- `EXPLAIN ANALYZE` esegue la query. Confrontate righe stimate ed effettive.

`guide.md` del plugin porta gli esercizi: interrogazioni, manipolazione dei dati, ruoli e privilegi, backup con `pg_dump`, configurazione.